COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	9/17/26	**DATE SUBMITTED:**	9/19/26
NAME:	Nichollo Dave A. Galvez	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

Table 8-1. Array of Values for Sort Algorithm Testing

Code

```cpp
#include <iostream>
#include "sortingAlgo.h"
#include <cstdlib>
#include <ctime>

const int maxSize = 100;

void displayArray(int arr[]);

int main() {
    int array[maxSize];

    std::srand(std::time(0));

    for (int i = 0; i < maxSize; i++){
        array[i] = std::rand() % 100;
    }

    std::cout << "Unsorted Array:\n";
    displayArray(array);

    /*
    std::cout << "\nShell Sort:\n";
    shellSort(array, maxSize);
    displayArray(array);
    */

    /*
    std::cout << "\nMerge Sort:\n";
    mergeSort(array, 0, maxSize - 1);
    displayArray(array);
    */


    std::cout << "\nQuick Sort:\n";
    quickSort(array, 0, maxSize - 1);
    displayArray(array);


}

void displayArray(int arr[]) {
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

	Unsorted Array: 10 68 38 43 98 91 40 64 34 95 36 32 36 42 67 37 53 17 59 4 79 38 91 69 89 30 43 92 8 58 20 23 11 73 71 65 65 30
Observation	

Table 8-2. Shell Sort Technique

	`//Shell Sort`
	`template <typename T>`
	`void shellSort (T arr[], int size) {`
	`  for (int i = size / 2; i > 0; i /= 2){`
	`  for (int j = i; j < size; j++){`
	`  T temp = arr[j];`
	`  int k;`
	`  for (k = j; k >= i && arr[k - i] > temp; k -= i){`
	`  arr[k] = arr[k - i];`
	`  }`
	`  arr[k] = temp;`
	`  }`
	`  }`
	`}`
	
	Shell Sort:
	1 1 1 3 4 7 7 7 9 9 10 10 12 14 15 17 18 19 19 20 20
	78 78 79 79 80 81 83 84 85 86 86 88 88 88 89 92 93

Observation	

Table 8-3. Merge Sort Algorithm

Code	`//helper function for merge sort`
	`template <typename T>`
	`void merge(T arr[], int left, int mid, int right) {`
	`  int n1 = mid - left + 1;`
	`  int n2 = right - mid;`
	
	`  T* L = new T[n1];`
	`  T* R = new T[n2];`
	
	`  for (int i = 0; i < n1; i++)`
	`  L[i] = arr[left + i];`
	`  for (int j = 0; j < n2; j++)`
	`  R[j] = arr[mid + 1 + j];`
	
	`  int i = 0;`
	`  int j = 0;`
	`  int k = left;`
	
	`  while (i < n1 && j < n2) {`
	`  if (L[i] <= R[j]) {`
	`  arr[k] = L[i];`
	`  i++;`
	`  } else {`
	`  arr[k] = R[j];`
	`  j++;`
	`  }`
	`  k++;`
	`  }`
	
	`  while (i < n1) {`
	`  arr[k] = L[i];`
	`  i++;`
	`  k++;`
	`  }`
	
	`  while (j < n2) {`
	`  arr[k] = R[j];`
	`  j++;`
	`  k++;`
	`  }`
	
	`  delete[] L;`
	`  delete[] R;`
	`}`

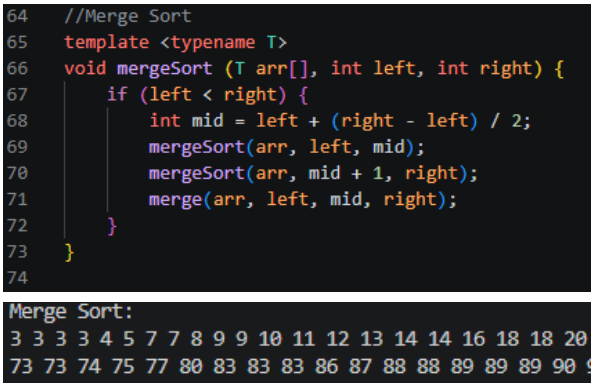
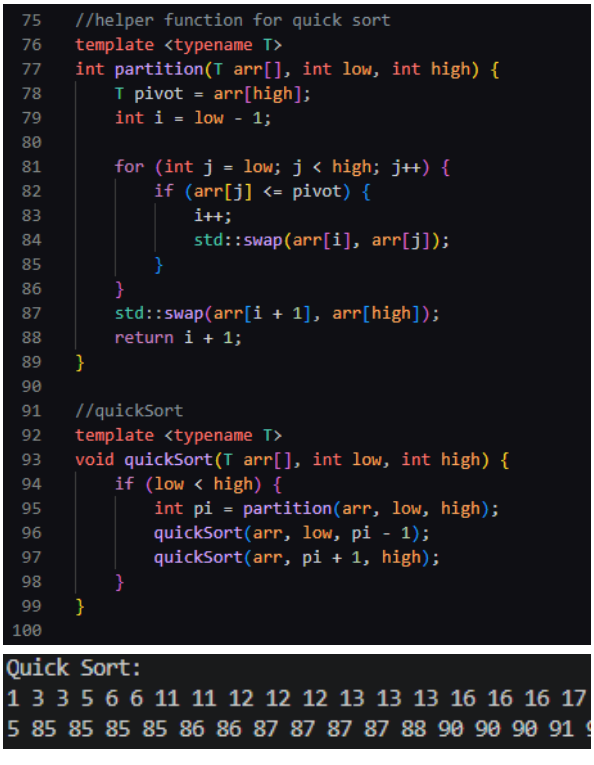
	
Observation	

Table 8-4. Quick Sort Algorithm

Code	
Observation	

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. **Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.**

```cpp
#include <iostream>
#include "problem1.h"

int main() {
    int arr[] = {67, 69, 10, 7, 23};
    int n = 5;

    std::cout << "Unsorted Array:\n";
    for (int i = 0; i < n; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << "\n\n";

    hybridquickSort(arr, 0, n - 1);

    std::cout << "Sorted Array:\n";
    for (int i = 0; i < n; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

```cpp
#ifndef PROBLEM1_H
#define PROBLEM1_H

// Partition function from Quick Sort
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            T temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    T temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

// Simple Selection Sort for sub-lists
template <typename T>
void selectionSort(T arr[], int start, int end) {
    for (int i = start; i < end; i++) {
        int minIdx = i;
        for (int j = i + 1; j <= end; j++) {
            if (arr[j] < arr[minIdx]) {
                minIdx = j;
            }
        }
        T temp = arr[i];
        arr[i] = arr[minIdx];
        arr[minIdx] = temp;
    }
}

// Partitions first, then sorts left and right sub-lists
template <typename T>
void hybridquickSort(T arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

```

```
47          // Sort sub-lists using selection sort
48          selectionSort(arr, low, pi - 1);
49          selectionSort(arr, pi + 1, high);
50      }
51  }
52
53  #endif
```

Explanation: You can sort the left and right sub-arrays using the partitioning method. During this process, placing elements around the pivot automatically moves them to their correct, final sorted positions.

2. **Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?**

```
1   #include <iostream>
2   #include "Problem2.h"
3
4   // Forward declaration matching the displayArray definition below
5   void displayArr(int arr[], int size);
6
7   int main() {
8       // create the array in the problem
9       int arr[15] = {1, 20, 23, 45, 50, 84, 16, 39,
10                     40, 69, 38, 67, 19, 58, 34};
11
12      // get the number of elements
13      int n = sizeof(arr) / sizeof(arr[0]);
14
15      std::cout << "Original Array:\n";
16      displayArr(arr, n);
17
18      // perform quick sort
19      quickSort(arr, 0, n - 1);
20
21      std::cout << "\nSorted Array:\n";
22      displayArr(arr, n);
23
24      return 0;
25  }
26
27  void displayArr(int arr[], int size) {
28      for (int i = 0; i < size; i++) {
29          std::cout << arr[i] << " ";
30      }
31      std::cout << std::endl;
32  }
```

```cpp
#ifndef PROBLEM2_H
#define PROBLEM2_H

// Helper partition function for Quick Sort
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high];
    int i = low - 1;


    // scans every number in the array for swapping
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            T temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    T temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

// quickSort
template <typename T>
void quickSort(T arr[], int low, int high) {

    //checker if low < high
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

#endif
```

```
Original Array:
1 20 23 45 50 84 16 39 40 69 38 67 19 58 34

Sorted Array:
1 16 19 20 23 34 38 39 40 45 50 58 67 69 84
```

Code Analysis:

In the header file, a partition helper function uses the final element as a pivot to move smaller values forward and place the pivot in its correct position. The quickSort function then recursively splits and sorts the left and right sub-arrays while low < high. In Problem2.cpp, I prototyped displayArr and defined a 15-element array in main(), calculating its length using sizeof(arr) / sizeof(arr[0]). The program prints the initial array, runs quickSort, and displays the sorted output.

Conclusion:

This activity provided hands-on experience with Merge, Shell, and Quick Sort algorithms. Working directly with the code illustrated how each algorithm handles array division and element comparison differently. I gained a clearer understanding of why Quick Sort is favored for its speed and in-place memory efficiency. Ultimately, this lab emphasized the importance of choosing a sorting algorithm based on data constraints and execution speed.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.